

INF221 - [Modern] Web Design & Development

Isaac S. Mwakabira

Learning Outcomes

By the end of this lecture, students will be:

- ❖ introduced to web frameworks & their key concepts

Introduction

In Modern Software Development

- ❖ Design vs Architecture

- ❖ Paradigms

 - Functional programming

 - Object Oriented Programming

 - Objects(Class), Abstraction, Inheritance - interfaces & abstract classes, Polymorphism, Encapsulation

- ❖ Design Principles

- ❖ Design Patterns

 - Structural, Creation, Behavioral

- ❖ Software Architecture Patterns - MVC

Programming Paradigms

Architecture begins with your code.

- ❖ How should we write our code so as to ensure good design architecture?

Paradigms, principles and patterns exist for this purpose.

- ❖ Paradigm
 - A category of entities sharing common characteristics.
 - Ways of programming that inform developers on what programming structure to use and when to use them.
- ❖ They streamline development process by describing tasks required for building high-quality software products.
- ❖ Structured, functional, & object oriented.

Objective:

- ❖ Structured development process, as guidance for quality software product

Programming Paradigms

❖ Structured Programming

- *Structured programming imposes discipline on direct transfer of control*

❖ Object Oriented Programming

- *Object-oriented programming imposes discipline on indirect transfer of control*
- Objects(Class), Abstraction, Inheritance - interfaces & abstract classes, Polymorphism, Encapsulation
- Classes are personified as Objects
 - Combination of:
 - Data
 - ◆ properties define value and
 - Functions/methods
 - ◆ behaviour
- Model the real world
 - Abstraction - Hidden. To what extent?
 - Encapsulation
 - hide data(public, private, protected)
 - expose functions(public, private, protected)
 - Inheritance - redeclaration of a group of variables and functions within an enclosing scope
 - Polymorphism - many shapes, Objects can have many shapes

Programming Paradigms

❖ Functional Programming

- *Functional programming imposes discipline upon assignment*

❖ Aspect Oriented Programming

- Complements OOD by providing another way of thinking about program structure
- key unit of modularity in OOP is the **class**, whereas in AOP the unit of modularity is the **aspect**
- Mostly used in Spring + related frameworks ie. Spring Boot

Principles - SOLID

Software SOLID principles tell developers or engineers how:

- ❖ To arrange our functions and data structures into classes, and
- ❖ Those classes are related and interconnected

Goals:

- ❖ Torate change
- ❖ Easy to understand software structures
- ❖ Software structures that are the basis of components that can be used in many software systems

List:

Single Responsibility Principle(SRP), Open-Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, Dependency Inversion Principle

Software Design Patterns

Solutions to general problems. Singular software development approach to software problem solving.

Proven and implemented in many software frameworks. Laravel, Ruby on Rails, etc.

Used by experienced object oriented software developers.

Types or Categories

- ❖ Structural -
- ❖ Creational - Builder, Singleton, Factory,
- ❖ Behavioral -

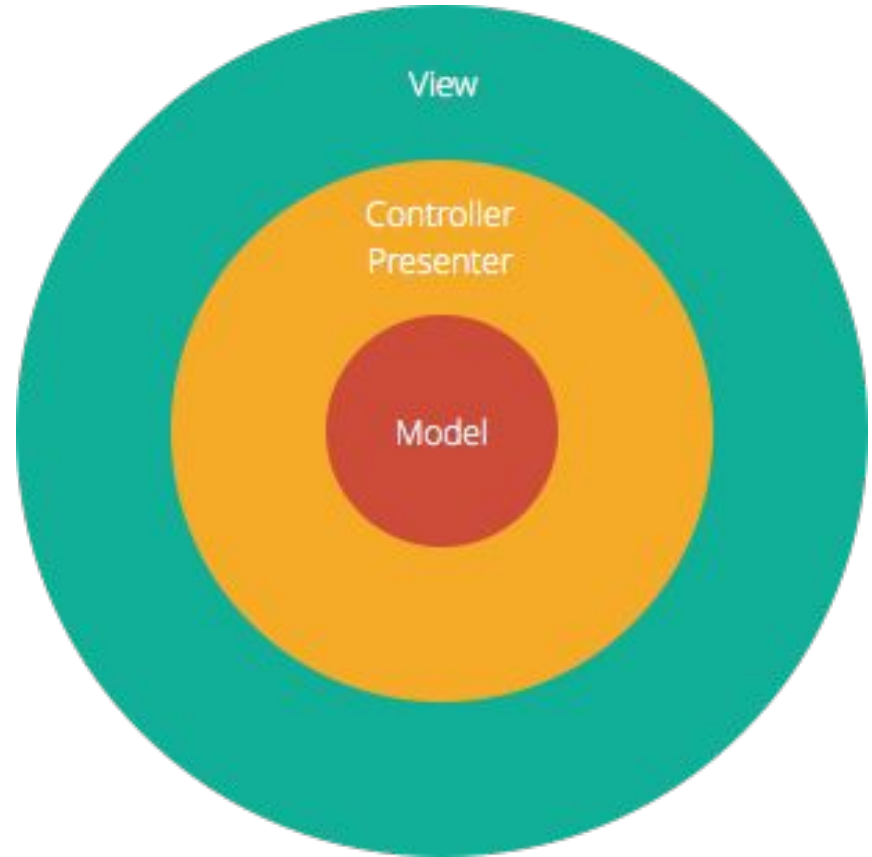
Design principles and patterns - are where software architectural patterns are built upon.

Architectural Patterns

- ❖ Model, View, Controller
- ❖ Model, View, Presenter
- ❖ Model View ViewModel/Binder

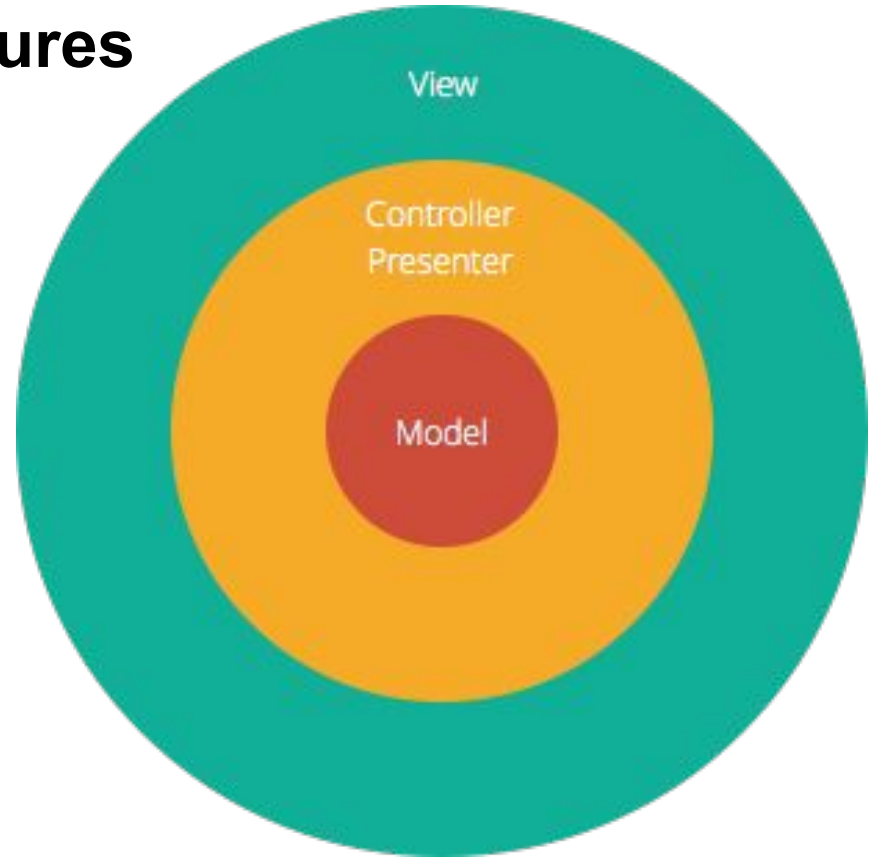
Model View Controller

- ❖ It is an architectural pattern
- ❖ Separates an application into 3 three main components
 - ie. Model, View, Controller
- ❖ It ensures the separation of concerns:
 - each architectural component handles specific development aspect within an application.
- ❖ Separates business logic from presentation logic
- ❖ Widely used in professional web applications &/or mobile apps.
- ❖ Laravel & many other frameworks enforce this pattern
 - Others includes Angular, Rails, Zend Framework, CodeIgniter, Laravel, Fuel PHP, etc.



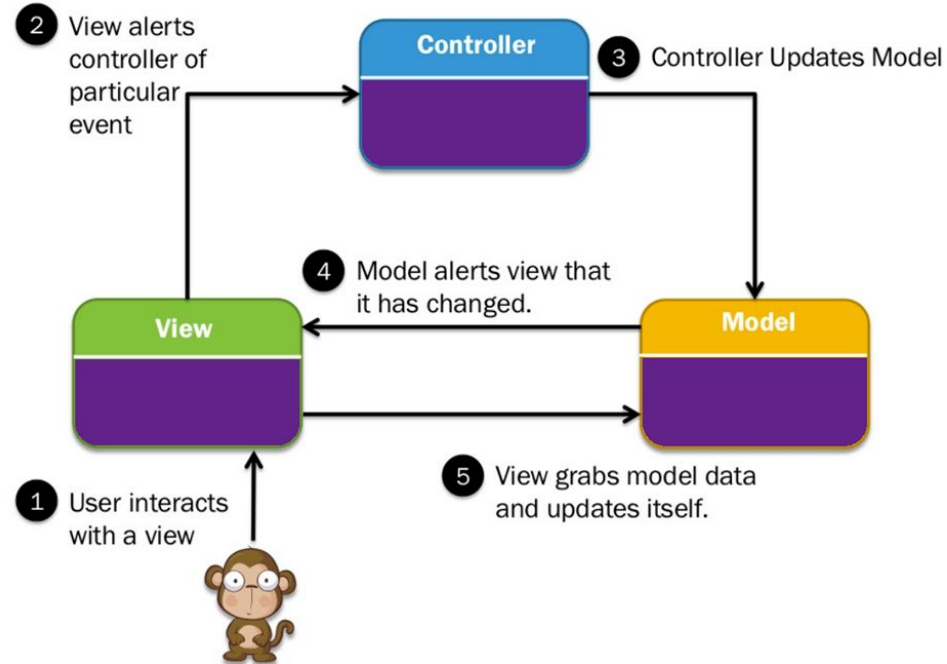
Model View Controller - Features

- ❖ Easy and frictionless testability.
- ❖ Highly testable, extensible and pluggable framework
- ❖ Offers full control over your HTML as well as your URLs
- ❖ Clear separation of logic:
 - Model, View, Controller.
 - Separation of application tasks viz. business logic, UI logic, and input logic
- ❖ URL Routing for SEO Friendly URLs.
 - Powerful URL - mapping for comprehensible and searchable URLs
- ❖ Supports for Test Driven Development (TDD)



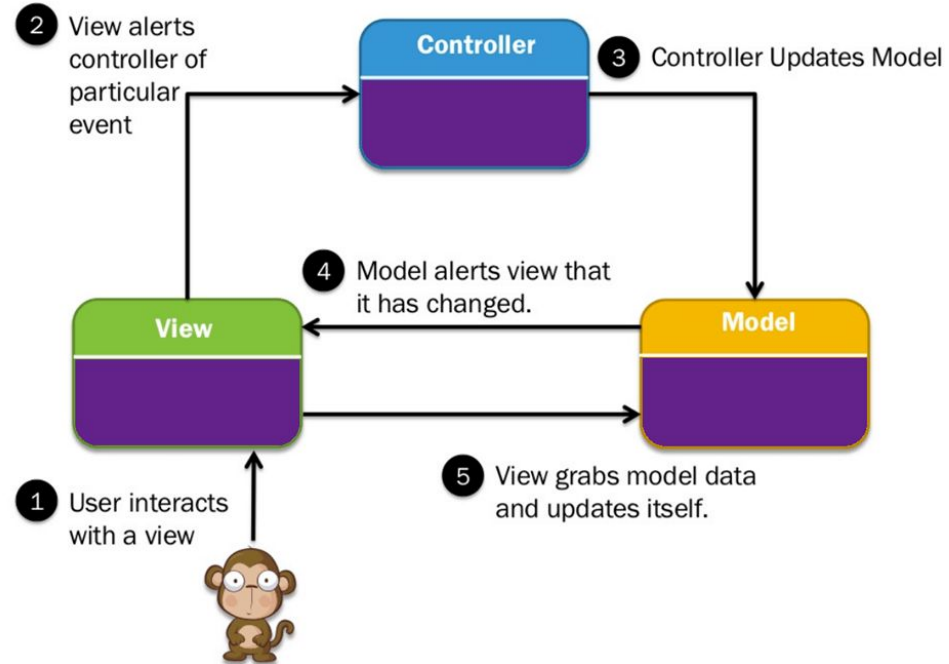
Model-View-Controller: **Model**

- ❖ Model component stores data and its related logic.
- ❖ It represents data that is being transferred between controller components or any other related business logic.
- ❖ For example, a Controller object will retrieve the customer info from the database. It manipulates data and send back to the database or use it to render the same data to the View.



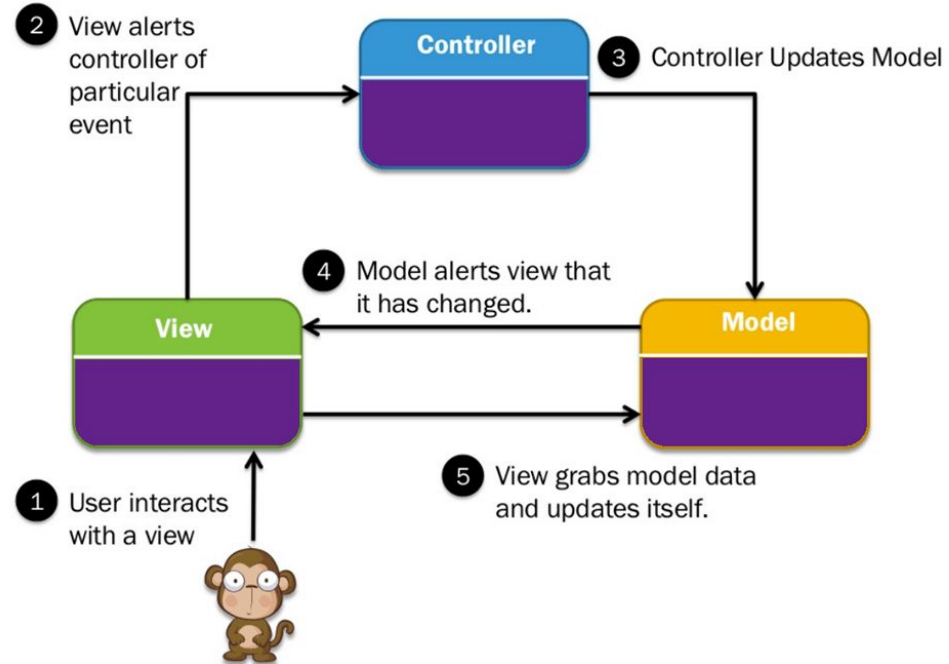
Model-View-Controller: **View**

- ❖ Presents data to the user or handles user interaction
- ❖ **Views** are created by the data collected from the **Model** data



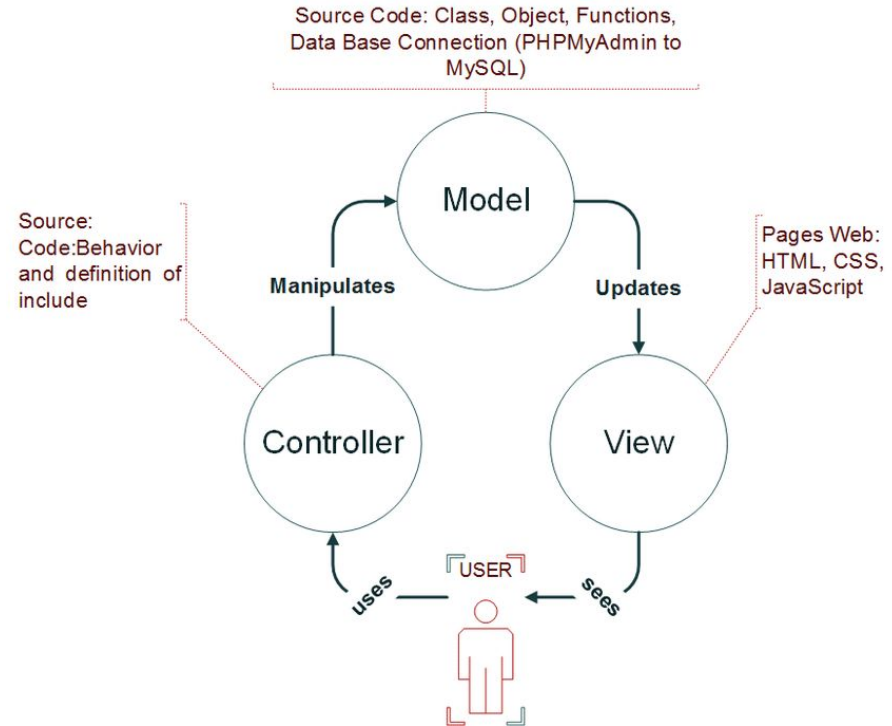
Model-View-Controller: Controller

- ❖ The Controller is that part of the application that handles the user interaction.
 - It interprets the mouse and keyboard inputs from the user, informing model and the view to change as appropriate.
- ❖ A Controller sends commands to the model to update its state
 - E.g., saving a specific document
- ❖ The controller also sends commands to its associated view to change the view's presentation
 - for example scrolling a particular document
- ❖ It responds to the request from the views and also responds to instructions from the controller to update itself.
 - It is also the lowest level of the pattern which is responsible for maintaining data.



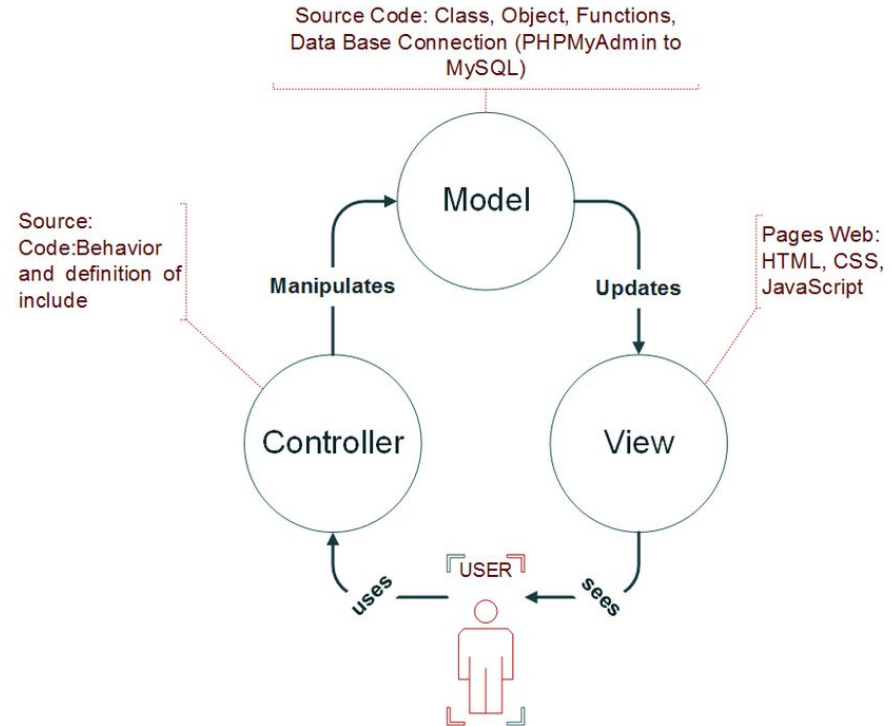
Model View Controller – Key Benefits

- ❖ Individual components can be tested separately
- ❖ Scalable - code can be easily managed
 - Easy code maintenance; easy to extend and grow
- ❖ Development of the various components can be performed in parallel
- ❖ It helps you to avoid complexity by dividing an application into the three units.
 - Model, view, and controller
- ❖ Offers the best support for test-driven development (TDD)
 - All classes and objects are independent of each other so that you can test them separately



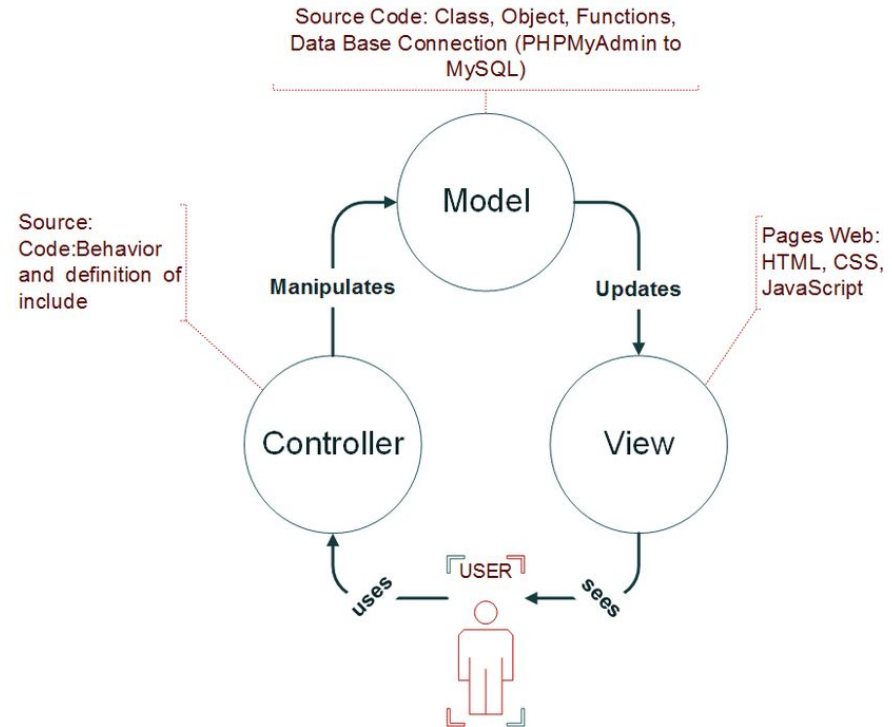
Model View Controller – Key Benefits

- ❖ It works well for web apps which are supported by large teams of web designers and developers
- ❖ Provides clean separation of concerns(SoC)
- ❖ Search Engine Optimization (SEO)
- ❖ It allows for logical grouping of related actions on a controller together
- ❖ It only uses a Front Controller pattern which process web application requests through a single controller



Model View Controller – Disadvantages

- ❖ Difficult to read, change, to unit test, and reuse this model
- ❖ The framework navigation can some time be complex as it introduces new layers of abstraction which requires users to adapt to the decomposition criteria of MVC
- ❖ No formal validation support
- ❖ Increased complexity and inefficiency of data
- ❖ Difficulty to use with modern user interfaces
- ❖ There is a need for multiple programmers to conduct parallel programming
- ❖ Knowledge of multiple technologies is required



Model View Controller – Summary

- ❖ The MVC is an architectural pattern that separates an application into:
 - Model
 - View and
 - Controller
- ❖ Model
 - It includes all the data and its related logic
- ❖ View
 - Present data to the user or handles user interaction
- ❖ Controller
 - An interface between Model and View components
- ❖ MVC is a highly testable, extensible and pluggable framework
- ❖ Some popular frameworks are Rails, Zend Framework, CodeIgniter, Laravel, Fuel PHP, etc.

Web Architecture vs Web Framework – Key Differences

“...an architecture is the the abstract design concept of an application...it offers a structure of moving parts and how they're connected...”

“...a framework is a pre-built general or special purpose architecture that's designed to be extended...”

An introduction to Software Frameworks

- ❖ By definition, a framework is an abstraction in which software providing generic functionality can be selectively extended:
 - through additional user-written code
 - thus providing application-specific software
- ❖ A software framework provides a standard way to build and deploy applications
- ❖ Software frameworks may include:
 - support programs, compilers, code libraries, tool sets, and application programming interfaces (APIs)
 - that bring together all the different components to enable development of a system.
- ❖ Software frameworks typically contain considerable housekeeping and utility code
 - in order to help bootstrap user applications
 - but generally focus on specific problem domains

Why A Framework?

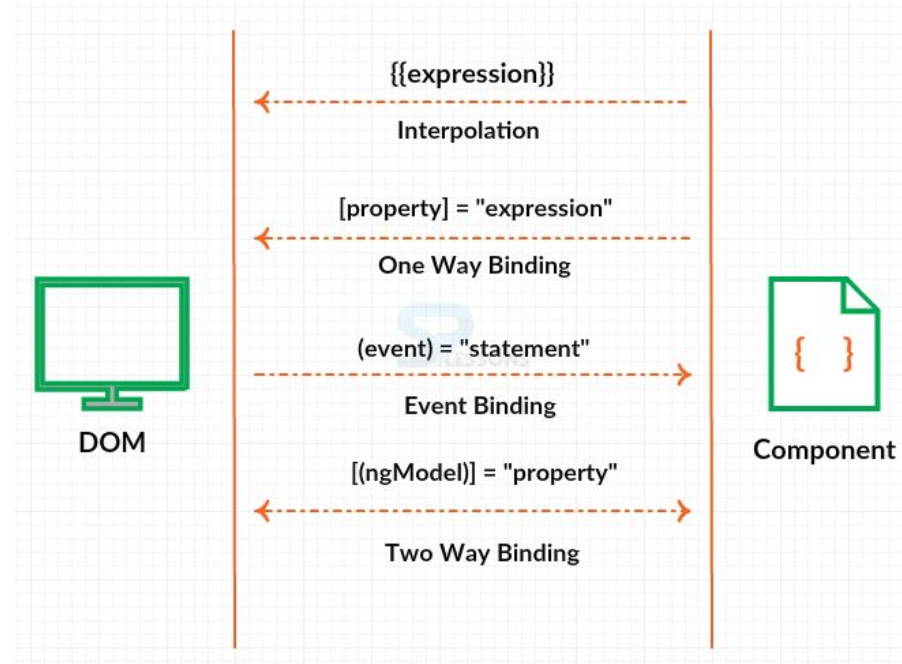
- ❖ Investing in the Task, Not in the Technology
- ❖ Guaranteed upgradability & maintenance
- ❖ Efficiency
- ❖ Integration
- ❖ Tooling
- ❖ Compartmentalization
- ❖ Routing – client-side and server-side

Examples

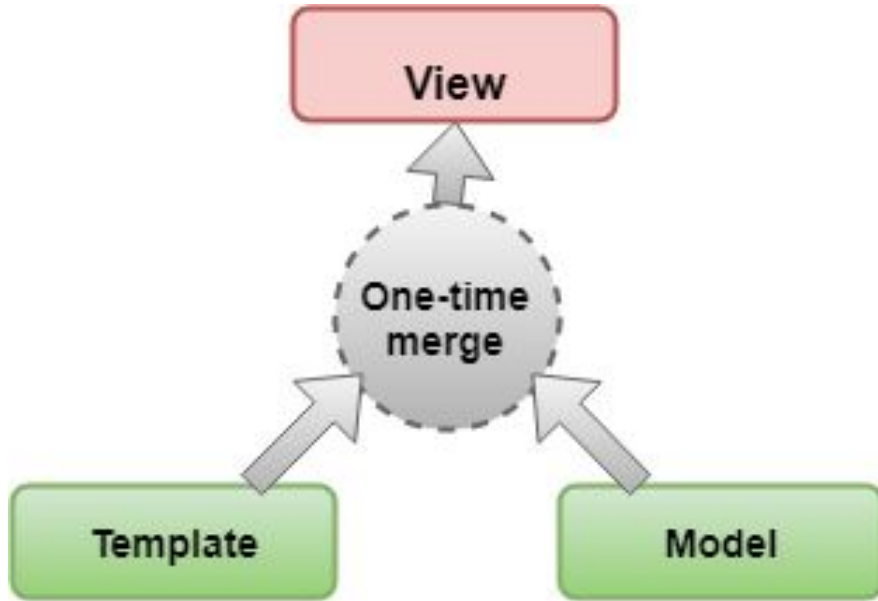
- ❖ Vue.js; React; Angular; Laravel; Spring or Spring Boot
- ❖ How is each different from the other, what philosophies does each advocate?
What concepts does each implement?

Frameworks & Concepts - Data Binding

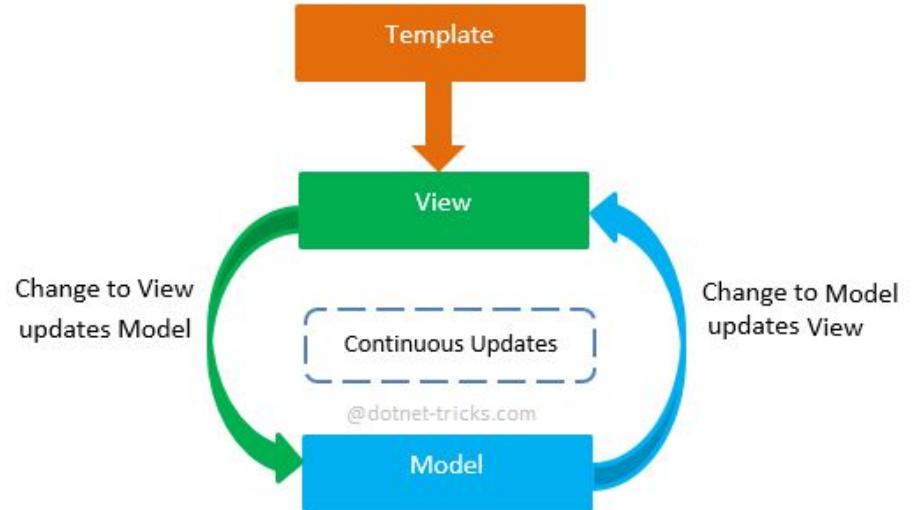
- ❖ Bind **UI** components in your **View** to data sources (**Model**) in your app
 - using a declarative format rather
 - ie. `<div>{{ 'data-here' }}</div>`
- ❖ Is a general technique that binds data source provider and consumer together and synchronizes them.
- ❖ This is usually done with two data/information sources with different languages
 - as in XML data and UI data
 - In UI data binding, data and information objects of the same language but different logic function are bound together (e.g. HTML UI elements to underlying objects).
 - In a data binding process, each data change is reflected automatically by the elements that are bound to it.
- ❖ The term data binding is also used in cases where an outer representation of data in an element changes, and the underlying data is automatically updated to reflect this change.
- ❖ As an example, a change in an input element could modify the underlying data value or object
- ❖ **One** way or **Two** way data binding ie in angularJS, React & Vue etc.



One vs. Two Way Data Binding



One-way data binding



Two-Way Data Binding

Spring [Spring Boot]

Key Concepts

Two foundational concepts upon which [Spring framework](#) is founded. It is based on two design principles –

Dependency Injection and Aspect Oriented Programming

- ❖ Dependency Injection (DI) [design](#) pattern
 - a pattern that allows to build very decoupled systems
 - spring uses this concept to wire the **classes** up by using an **XML** file or [annotations](#)
 - this way all the **objects** are **instantiated** and **initialized** by Spring and **injected** in the **right places**
- ❖ Inversion of Control (IoC) [principle](#)
 - is the process of providing the dependencies
 - it manages java objects from instantiation to destruction
 - the IoC container manages bean scope, lifecycle, and all AOP features for which the framework is configured and coded

```
1 package com.gradle.sales.service.services;
2
3 import java.util.List;
4 import java.util.Optional;
5 import java.util.stream.Collectors;
6
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.hateoas.CollectionModel;
9 import org.springframework.hateoas.EntityModel;
10 import org.springframework.stereotype.Component;
11
12 import com.gradle.sales.service.common.ProductNotFoundException;
13 import com.gradle.sales.service.data.repository.ProductRepository;
14 import com.gradle.sales.service.domain.model.Product;
15 import com.gradle.sales.service.helper.ProductResourceAssembler;
16
17 @Component
18 public class ProductServiceImpl extends ProductService {
19
20     @Autowired
21     private ProductRepository productRepository;
22
23     @Autowired
24     private ProductResourceAssembler assembler;
25
26     public ProductServiceImpl() {}
27
28     @Override
29     public EntityModel<Product> findById(Long id) {
30         return assembler.toModel(productRepository.findById(id)
31                                 .orElseThrow(() -> new ProductNotFoundException(id)));
32     }
33 }
```

Spring Annotations

- ❖ allows developers to configure dependencies and implement dependency injection programmatically - java code

- The framework implements and promotes the principle of control inversion (IOC) or dependency injection (DI) and is in fact an IOC container.
- Traditionally, Spring allows a developer to manage bean dependencies by using XML-based configuration.
 - There is an alternative way to define beans and their dependencies. This method is a Java-based configuration.
 - Unlike the XML approach, Java-based configuration allows you to manage bean components programmatically. That's why Spring annotations were introduced.

```
1 package com.gradle.sales.service.controller;
2
3 import ...
4
27 @RestController
28 public class OrderController {
29
30
31     private final OrderRepository orderRepository;
32     private final OrderResourceAssembler assembler;
33
34     @Autowired
35     public OrderController(OrderRepository orderRepository, OrderResourceAssembler assembler) {
36         this.orderRepository = orderRepository;
37         this.assembler = assembler;
38     }
39
40     @RequestMapping(value = "/orders",
41                     method = RequestMethod.GET,
42                     produces = {
43                         MediaType.APPLICATION_JSON_VALUE,
44                         MediaType.APPLICATION_XML_VALUE
45                     })
46     public CollectionModel<EntityModel<Order>> all() {
47
48         List<EntityModel<Order>> orders = orderRepository.findAll() List<Order>
49         .stream() Stream<Order>
50         .map(assembler::toModel) Stream<EntityModel<Order>>
51         .collect(Collectors.toList());
52
53         return assembler.toResources(orders);
54     }
55
56     @RequestMapping(value = "/orders/{id}",
```

```
19 import Lombok.;
20
21 /**
22  * @author Shukrani Isaac Mwakabira (@Link mshukran@gmail.com)
23  */
24 @Data
25 @Setter
26 @Getter
27 @AllArgsConstructor
28 @NoArgsConstructor
29 @Entity
30 @Table(name = "orders")
31 @JsonIgnore(as = Order.class)
32 @Builder(toBuilder = true)
33 public class Order {
34     @GeneratedValue(strategy = GenerationType.AUTO)
35     @JsonInclude(JsonInclude.Include.NON_NULL)
36     private @JsonIgnore @Id Long id;
37     private @JsonInclude(JsonInclude.Include.NON_NULL) String description;
38     private @JsonInclude(JsonInclude.Include.NON_NULL) OrderStatus status;
39     private @JsonInclude(JsonInclude.Include.NON_NULL) String uid;
40     private @JsonInclude(JsonInclude.Include.NON_NULL) String name;
41     private @JsonInclude(JsonInclude.Include.NON_NULL) String displayName;
42     private @JsonInclude(JsonInclude.Include.NON_NULL) Date created, lastUpdated;
43
44     @ManyToOne(cascade = CascadeType.ALL, mappedBy = "orders")
45     @JsonInclude(JsonInclude.Include.NON_NULL)
46     private Collection<Product> products;
47 }
```

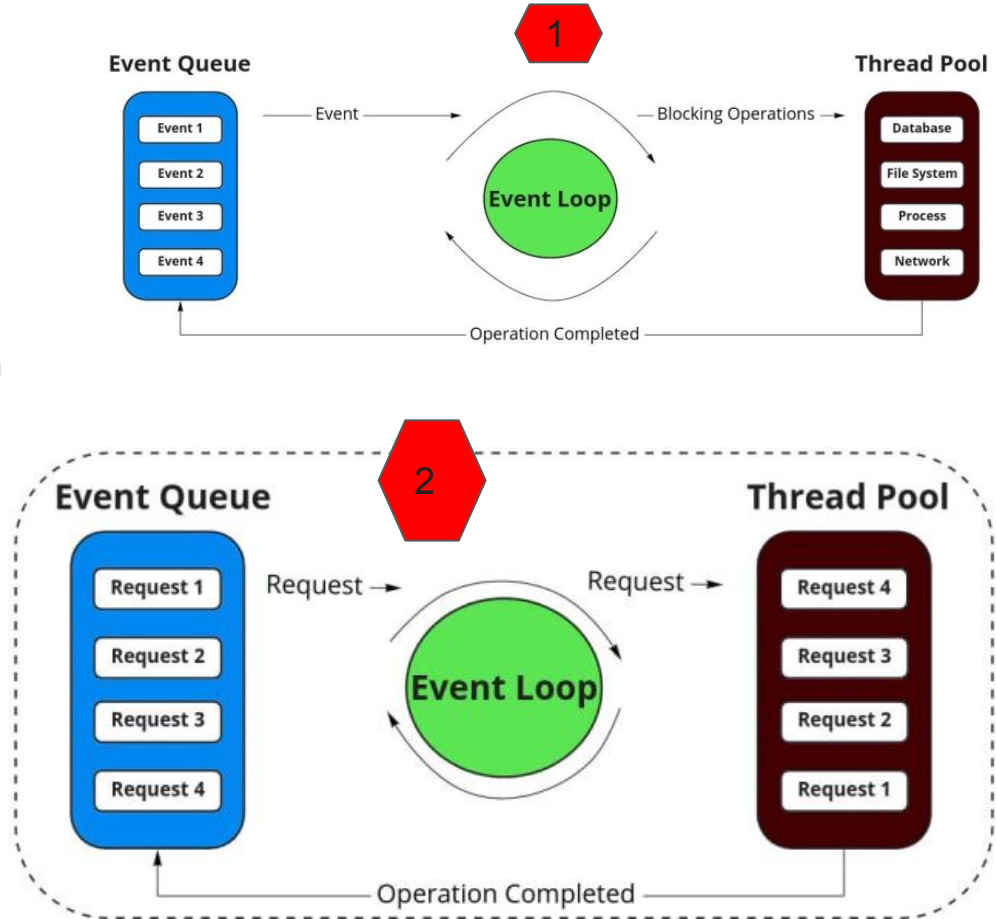
Node [JS]

Key Concepts

- ❖ is an open-source, cross-platform, back-end JavaScript runtime environment
 - runs on V8 engine
 - executes JavaScript code outside a web browser
- ❖ the engine that compiles JavaScript in a web Browser is the same engine that runs Node.js in it's core,
 - and this is what makes it so fast, especially for web applications
 - server-side platform to develop modern, reliable and scalable web applications
- ❖ characteristics
 - Non-blocking I/O
 - Single threaded
 - Event driven
 - No buffering
 - Scalable
 - Cross-platform environment and
 - can run on Microsoft Windows Linux



-Requests ➤
-Requests ➤
-Requests ➤
-Requests ➤



Vue & React [cf. NextJS]

Key Concepts

❖ Virtual DOM Explained

- HTML DOM manipulation is expensive
 - As application scales – becomes hard to update
- How does it relate to dirty model checking, in Angular?
 - Any small change in the data updates the whole model, hence expensive

❖ No direct DOM manipulation

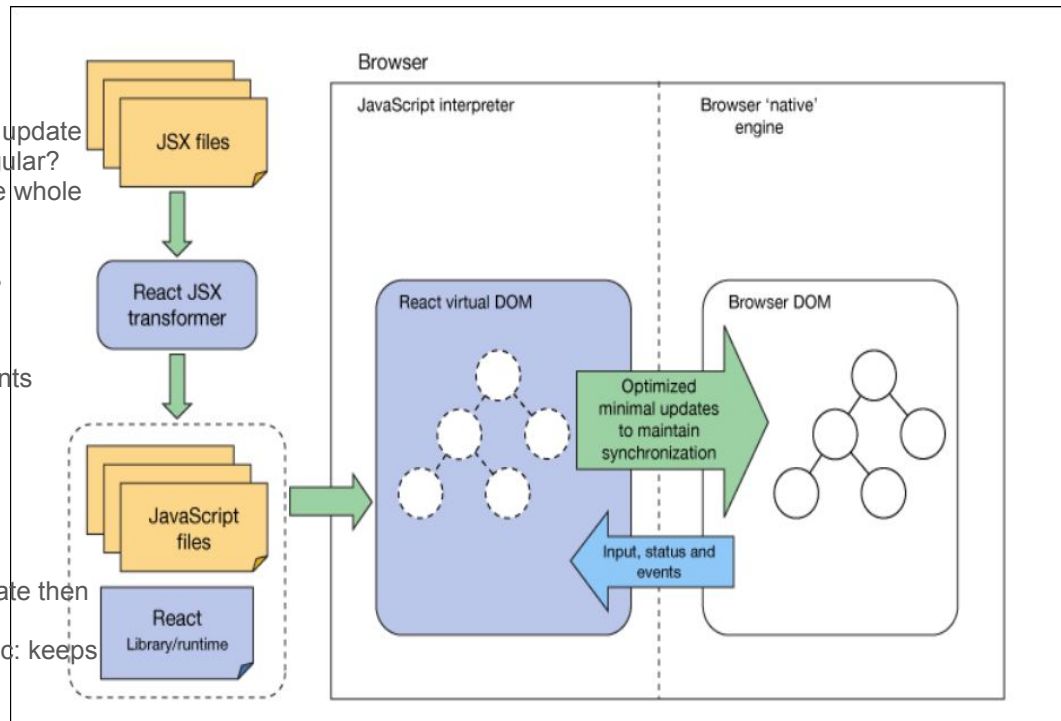
- How do we avoid manipulating the DOM directly?
- Why is it considered faster than direct DOM manipulation?
- Keep an in-memory representation of html elements
- Virtual Document Object Model

❖ JSX – Javascript eXtension – DSL (Domain-specific-language)

❖ Declarative

❖ Component based

- encapsulated components - manage their own state then compose them to make complex UIs
- templates are not used, Javascript does the magic: keeps state out of the DOM
- every component maintains its own internal state



References

❖ <https://www.journaldev.com/16966/spring-annotations>

